ClaimLine AI

Voice-Based Insurance Claims Intake and Triage Agent — Project Specification

Muhammad Ali Hashim

1. Executive Summary

ClaimLine AI is a voice-based insurance claims intake and triage system built on a **Retell Conversation Flow agent**, backed by **two n8n automation workflows**, a **Google Sheets** data layer, and a **Streamlit** dashboard where the live agent is embedded and claim data is visualized in real time.

A caller can: ask general policy questions (answered from an uploaded Knowledge Base), file a new claim (validated against a policy database, triaged for urgency, and logged automatically), or check the status of an existing claim by ID — all through natural conversation, with no manual data entry on the business side.

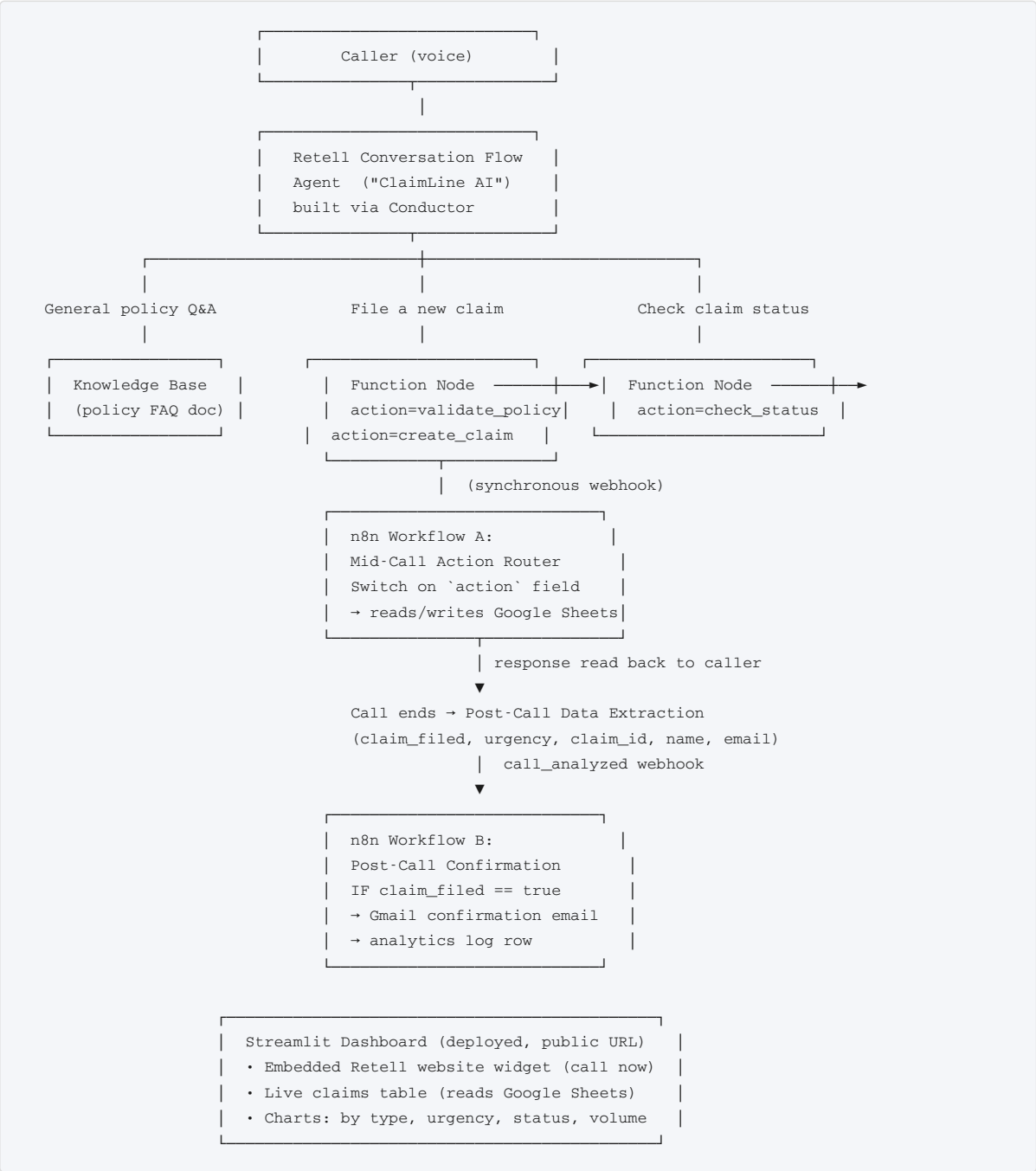
This is a deliberate step up from the previous BrightPath Tutoring project: it moves from a **Single Prompt** agent to a **Conversation Flow** agent (structured, branching, node-based — built using Retell's Conductor AI copilot), and from a single linear n8n workflow to **two coordinated workflows** — one synchronous (mid-call actions) and one asynchronous (post-call follow-up) — including conditional routing inside n8n itself.

2. Objectives & Skills Demonstrated

- Designing a **multi-intent, branching conversation flow** (not a single linear script) using Retell's node-based builder and Conductor AI.
- Mid-call **synchronous function calls** from the agent to an external system (n8n) that return data the agent must react to differently depending on the result (valid vs invalid policy, found vs not-found claim).

- **Conditional/triage logic**: classifying incoming claims as urgent vs standard based on conversation content, and branching the flow accordingly.
- **Post-call automation** decoupled from the live call (confirmation emails, logging) — reinforcing the sync-vs-async architecture pattern from the previous project.
- Treating **Google Sheets as a lightweight relational data layer** (two related tables: Policies and Claims) accessed from both n8n and a separate Python dashboard.
- Building and **deploying a public-facing production demo** (Streamlit Community Cloud) that embeds a real, callable voice agent — something a recruiter or interviewer can interact with directly, not just read about.
- Working entirely within **free tiers** — no paid API keys anywhere in the stack.

3. System Architecture



Design rationale: the live call only ever waits on one synchronous round-trip to n8n (Workflow A) to validate a policy, create a claim, or look up a status — kept fast so the caller isn't left in

silence. Everything non-urgent (emailing, analytics logging) is pushed to Workflow B, which only fires after the call ends. This mirrors real production voice-AI systems, where call latency budget is protected aggressively.

4. Tech Stack (all free-tier)

Layer	Tool	Free-tier basis
Voice agent	Retell AI (Conversation Flow + Conductor)	Free trial credits
Mid-call & post-call automation	n8n	Free trial (already available to you)
Data layer	Google Sheets	Free Google account
Email	Gmail (via n8n's Gmail node)	Free Google account
Dashboard & embed host	Streamlit Community Cloud	Free public app hosting
Charts	Plotly / Altair (via Streamlit)	Open source
Sheets access from Python	`gspread` + a Google Service Account	Free (service accounts don't require billing for Sheets API at this scale)
Web call embed	Retell Website Widget (`<script>` tag + public key)	Included with Retell account, no backend proxy needed

5. Data Model — Google Sheets

Spreadsheet: `ClaimLine-Data`

Tab: `Policies` (pre-seeded, acts as the "database" of existing customers)

Column	Example
policy_number	POL-10234
holder_name	Sara Ahmed
coverage_type	Auto
valid_until	2027-01-01

Tab: `Claims` (populated automatically by the agent)

Column	Example
claim_id	CLM-000042
policy_number	POL-10234
holder_name	Sara Ahmed
incident_type	Accident
incident_date	2026-08-04
description	Rear-ended at a traffic signal, no injuries
urgency	Standard / Urgent
status	Filed

email	sara@example.com
created_at	2026-08-06 14:03

Tab: `Analytics` (populated by n8n Workflow B, powers the dashboard's volume chart)

Column	Example
timestamp	2026-08-06 14:03
claim_filed	true / false
urgency	Standard / Urgent / n/a
incident_type	Accident

5.1 Provided Seed Data & Knowledge Base Files

These files are provided alongside this specification and should be used as-is, not regenerated:

File	Purpose
`Policies-Seed-Data.csv`	10 fictional pre-existing policies (Auto, Health, Home, Life) — import this into the `Policies` tab so `validate_policy` has real data to check against.
`Claims-Template.csv`	Column headers plus one example row for the `Claims` tab — import this to get the correct columns before the agent starts appending real rows.
`Analytics-Template.csv`	Column headers plus one example row for the `Analytics` tab.
`ClaimLine-Policy-FAQ.docx`	The Knowledge Base document — upload this directly into Retell's Knowledge Base and attach it to the agent. Covers coverage types, how to file a claim, required documents, processing timeline, and status values.

6. Retell Conversation Flow Specification

Build using **Conductor** (Retell's in-dashboard AI copilot) rather than manual node-by-node placement — describe each section in plain English and let Conductor scaffold the nodes, then refine manually.

Global settings

- Voice: any neutral, professional voice.
- Knowledge Base: `ClaimLine-Policy-FAQ.docx` (coverage types, what documents are needed to file a claim, how long processing takes, general claims-process explanation).

Flow outline

1. **Start / Greeting node** — greets the caller, asks how it can help.
2. **Branch node — intent classification**: routes to one of three paths based on what the caller says (general question / file a claim / check status).
3. **Path A — General Q&A**: Conversation node(s) answering from the Knowledge Base, then a branch back to "anything else?" or end.

4. **Path B — File a new claim**:
    a. Conversation node: ask for policy number.
    b. Function node → n8n, `action: "validate_policy"`.
    c. Branch node on the result: if invalid → explain and offer to try again or transfer to a human; if valid → continue.
    d. Conversation node(s): collect incident type, date, description, name, email.
    e. Branch/logic node: classify urgency (rule-based — keywords like “injury”, “fire”, “hospital” → Urgent; otherwise Standard). This can be done with a **Code node** (simple keyword-matching logic) rather than relying purely on prompt judgment, which is more predictable — this is one of the specific advantages of Conversation Flow over Single Prompt.
    f. Function node → n8n, `action: "create_claim"` (sends all collected fields + urgency).
    g. Conversation node: read the returned claim ID back to the caller, confirm next steps.
    h. If urgency = Urgent → additional node informing the caller their claim is being prioritized (optionally a Transfer Call node if you want to demonstrate escalation, pointed at any test number, clearly labeled as a demo).
5. **Path C — Check claim status**:
    a. Conversation node: ask for the claim ID.
    b. Function node → n8n, `action: "check_status"`.
    c. Branch: found → read back status; not found → apologize and offer to try again or connect to a human.
6. **End node** on every path, with a warm closing line.

Post-Call Data Extraction fields

Field	Type	Description
`claim_filed`	Boolean	true if a new claim was successfully created this call
`claim_id`	Text	the generated claim ID, if any
`urgency`	Text	Standard / Urgent / not applicable
`name`	Text	caller’s name, if a claim was filed
`email`	Text	caller’s email, if a claim was filed

Webhook

- `call_analyzed` event → n8n Workflow B’s webhook URL.

7. n8n Workflow Specifications

Workflow A — Mid-Call Action Router (synchronous)

Triggered directly by the Retell Function node **during** the call, so it must respond quickly.

```
Webhook (POST) → Switch node on `action` field
  ├─ action = validate_policy → Google Sheets (lookup in Policies by policy_number)
  │                             → Respond to Webhook: { valid: true/false, holder_name }
  ├─ action = create_claim    → Code node: generate claim_id (e.g. CLM- + timestamp/counter)
  │                             → Google Sheets (append row to Claims)
  │                             → Respond to Webhook: { claim_id }
  └─ action = check_status    → Google Sheets (lookup in Claims by claim_id)
```

```
                                → Respond to Webhook: { found: true/false, status, incident_type }
```

Workflow B — Post-Call Confirmation & Logging (asynchronous)

Triggered by Retell's `call_analyzed` webhook, after the call has already ended.

```
Webhook (POST) → IF claim_filed == true
                    ├─ true  → Gmail: send claim confirmation email (includes claim_id)
                    │           → (optional) Google Sheets: append a row to an "Analytics" tab
                    │             logging call outcome for the dashboard's volume chart
                    └─ false → no action (or log a "no claim filed" analytics row)
```

8. Streamlit Dashboard Specification

A single-page Streamlit app, deployed publicly, containing:

1. **Header + embedded voice widget** — the Retell Website Widget script tag (public key + agent ID), rendered via `st.components.v1.html()`, so a visitor can click and literally talk to the agent from the deployed page. No backend token server needed since the widget uses the public-key flow.
2. **Live claims table** — reads the `Claims` tab from Google Sheets via `gspread` (authenticated with a Google Service Account whose credentials are stored in Streamlit's secrets manager, never committed to the repo).
3. **Charts**:
   - Bar chart: claims by `incident_type`.
   - Pie chart: `urgency` split (Standard vs Urgent).
   - Line chart: claims filed over time (by `created_at` date).
4. **Claim lookup box** — type a claim ID, see its details pulled live from the sheet (a read-only mirror of what the voice agent does).
5. Auto-refresh or a manual "Refresh data" button (avoid hammering the Sheets API — cache with a short TTL, e.g. 30–60 seconds).

9. Deployment Plan

1. Push the Streamlit app to a public GitHub repo (same pattern as the BrightPath project).
2. Deploy via **Streamlit Community Cloud** (free), pointing at the repo.
3. Store the Google Service Account JSON and the Retell public key as **Streamlit secrets** (`st.secrets`), never hardcoded or committed.
4. Confirm the deployed public URL loads the widget and the dashboard correctly before sharing it.

10. Testing Plan

In Retell Playground, before deployment: - Ask 2–3 policy/coverage questions → verify answers come from the Knowledge Base only. - File a claim with a valid policy number → verify a claim ID is read back and a row appears in the `Claims` tab. - File a claim with an invalid policy number → verify the agent handles it gracefully. - File a claim describing an injury → verify it's classified Urgent. - Check status with a real claim ID → verify correct status is read back. - Check

status with a fake claim ID → verify graceful “not found” handling.

After deployment: - Open the public Streamlit URL, use the embedded widget to make a real call, and verify the dashboard updates with the new claim after refreshing.

11. Constraints

- No paid API keys anywhere in the stack — every tool listed in Section 4 is used within its free tier.
- No real customer or insurance data — all policy and claim data is fictional/seeded for demo purposes, and this should be stated clearly in the deployed app and the README.

12. Deliverables Checklist

- ☐ Retell Conversation Flow agent, built via Conductor, covering all three paths in Section 6.
- ☐ Knowledge Base document (`ClaimLine-Policy-FAQ.docx`).
- ☐ Google Sheet with `Policies`, `Claims`, and `Analytics` tabs.
- ☐ n8n Workflow A (mid-call router) and Workflow B (post-call confirmation), both published/active.
- ☐ Streamlit dashboard with embedded widget, live table, and charts.
- ☐ Deployed public Streamlit URL.
- ☐ GitHub repository with full documentation, screenshots, and a README following the same structure as the BrightPath project.